Milestone-3 Report

RISC-V Assembly Implementation of
Linear and Extended Kalman Filters

Motion Capture Data Filtering

RISC-V LP64D ABI · IEEE-754 Double Precision

April 19, 2026

Contents

1 Introduction

Milestone-3 re-implements the core arithmetic kernels of the Linear Kalman Filter (LKF) and Extended Kalman Filter (EKF) from Milestone-2 in RISC-V scalar assembly (LP64D ABI, double-precision IEEE-754). The C driver files (`lkf_driver.c` and `ekf_driver.c`) handle dataset I/O, large matrix multiplications (276×276), and high-level filter logic, while all per-element and per-joint operations are delegated to the corresponding assembly source (`lkf_asm.s`, `ekf_asm.s`).

1.1 Files Provided

File	Purpose
`lkf_asm.s`	LKF assembly: 8 functions
`ekf_asm.s`	EKF assembly: 14 functions (superset of LKF)
`lkf_driver.c`	LKF C driver, dataset loader, CSV output
`ekf_driver.c`	EKF C driver, innovation, Joseph covariance
`Makefile`	Cross-compiles with `riscv64-linux-gnu-gcc`
`verify.py`	Numerical comparison vs Milestone-2
`plot_m3.py`	Milestone-3 plot generation
`plot_compare.py`	Milestone-2 vs 3 comparison plots

2 Function-by-Function Walkthrough

2.1 Shared Primitives (both `lkf_asm.s` and `ekf_asm.s`)

2.1.1 `mat_add_asm` — Element-wise Addition

```
# mat_add_asm(double *C, const double *A, const double *B, int
    n)
mat_add_asm:
    beqz   a3, mat_add_done       # n==0 -> skip
    mv     t0, a3                 # loop counter
mat_add_loop:
    fld    ft0, 0(a1)             # load A[i]
    fld    ft1, 0(a2)             # load B[i]
    fadd.d ft2, ft0, ft1          # ft2 = A[i]+B[i]
    fsd    ft2, 0(a0)             # store to C[i]
    addi   a0, a0, 8              # advance C pointer
    addi   a1, a1, 8              # advance A pointer
    addi   a2, a2, 8              # advance B pointer
    addi   t0, t0, -1             # decrement counter
    bnez   t0, mat_add_loop
mat_add_done:
    ret
```

Listing 1: = A[i] + B[i] for n elements]**`mat_add_asm`**: C[i] = A[i] + B[i] for n elements

Explanation. The function accepts three `double` pointers and a length `a3`. Each iteration loads one element from `a1` and `a2` into caller-saved floating-point registers `ft0`/`ft1`, performs `fadd.d`, and stores the result through `a0`. All three pointers are post-incremented

by 8 bytes (size of a `double`). No stack frame is required because only caller-saved registers are used.

2.1.2 `mat_sub_asm` — Element-wise Subtraction

Identical structure to **`mat_add_asm`**, replacing `fadd.d` with `fsub.d`. Computes $C_i = A_i - B_i$ in-place. Used in the covariance Joseph update $(I - KH)$.

2.1.3 `mat_scale_asm` — In-place Scalar Multiply

```
# mat_scale_asm(double *A, double scalar, int n)
#   scalar -> fa0,  n -> a1
mat_scale_asm:
    beqz  a1, mat_scale_done
    mv    t0, a1
mat_scale_loop:
    fld   ft0, 0(a0)
    fmul.d ft0, ft0, fa0        # ft0 *= scalar
    fsd   ft0, 0(a0)
    addi  a0, a0, 8
    addi  t0, t0, -1
    bnez  t0, mat_scale_loop
mat_scale_done:
    ret
```

Listing 2: *= scalar]**`mat_scale_asm`**: A[i] *= scalar

Explanation. The scalar is passed via the floating-point argument register `fa0` (LP64D convention). The loop loads, multiplies in-place, and stores, advancing the pointer by 8 bytes each iteration. Used for covariance process noise scaling.

2.1.4 `mat_dot_asm` — Dot Product

```
# mat_dot_asm(const double *A, const double *B, int n) ->
    double
mat_dot_asm:
    fmv.d.x fa0, zero           # accumulator = 0.0
    beqz    a2, mat_dot_done
    mv      t0, a2
mat_dot_loop:
    fld     ft0,  0(a0)
    fld     ft1,  0(a1)
    addi    a0,   a0, 8
    addi    a1,   a1, 8
    fmadd.d fa0,  ft0, ft1, fa0 # fa0 += A[i]*B[i]
    addi    t0,   t0, -1
    bnez    t0,   mat_dot_loop
mat_dot_done:
    ret
```

Listing 3: **`mat_dot_asm`**: $\sum A_i B_i$, result in fa0

Explanation. Uses `fmadd.d` (fused multiply-add) to accumulate $\sum_i A_i B_i$ into `fa0` with a single rounding per iteration, minimising floating-point error. The accumulator is initialised to +0.0 via `fmv.d.x fa0, zero`.

2.1.5 `vec_axpy_asm` — AXPY Operation

Implements $y_i \mathrel{+}= \alpha\, x_i$ for $i = 0 \ldots n-1$ using `fmadd.d fa0, ft0, ft1, fa0` per element. The scalar α is held in `fa0`; no stack frame is needed.

2.1.6 `predict_x_asm` — Constant-Jerk State Propagation

```
# predict_x_asm(double *x12, double dt)
predict_x_asm:
    addi  sp, sp, -32
    sd    ra, 24(sp)
    fsd   fs0, 16(sp)    # save callee-saved FP regs
    fsd   fs1,  8(sp)
    fsd   fs2,  0(sp)

    fmv.d  fs0, fa0              # fs0 = dt
    fmul.d fs1, fs0, fs0
    fmul.d fs1, fs1, ft6         # fs1 = dt^2/2  (ft6 = 0.5
       from rodata)
    fmul.d fs2, fs1, fs0
    fmul.d fs2, fs2, ft6         # fs2 = dt^3/6  (ft6 = 1/3
       from rodata)

    li    t6, 0
pxa_axis:                        # loop over 3 axes (b=0,1,2)
    bge   t6, t5, pxa_done
    slli  t0, t6, 5              # offset = b*32 bytes (4
       doubles)
    add   t0, a0, t0
    fld   ft0,  0(t0)            # pos
    fld   ft1,  8(t0)            # vel
    fld   ft2, 16(t0)            # acc
    fld   ft3, 24(t0)            # jerk
    fmadd.d ft0, fs0, ft1, ft0   # pos += dt*vel
    fmadd.d ft0, fs1, ft2, ft0   #      + dt^2/2*acc
    fmadd.d ft0, fs2, ft3, ft0   #      + dt^3/6*jerk
    fsd   ft0,  0(t0)
    fmadd.d ft1, fs0, ft2, ft1   # vel += dt*acc + dt^2/2*jerk
    fmadd.d ft1, fs1, ft3, ft1
    fsd   ft1,  8(t0)
    fmadd.d ft2, fs0, ft3, ft2   # acc += dt*jerk
    fsd   ft2, 16(t0)
    addi  t6, t6, 1
    j     pxa_axis
pxa_done:
    ...                          # restore callee-saved, ret
```

Listing 4: **`predict_x_asm`**: constant-jerk transition for one joint

Explanation. The 12-state vector per joint stores [pos, vel, acc, jerk] for each of 3 axes, laid out at byte offsets $0, 8, 16, 24$ (axis 0), $32 \ldots 56$ (axis 1), $64 \ldots 88$ (axis 2). The axis loop index `t6` is shifted left by 5 (`slli t0,t6,5`) to multiply by 32 (4 doubles $\times$ 8 bytes). Three callee-saved FP registers `fs0`/`fs1`/`fs2` hold dt, $dt^2/2$, and $dt^3/6$ across iterations to avoid redundant recomputation.

2.2 LKF-only: `lkf_asm.s`

2.2.1 `update_x_asm` — Kalman Gain Correction

```
# update_x_asm(double *x12, const double *K_row3, const double
    *innov3)
update_x_asm:
    fld   ft3,  0(a2)    # innov[0]
    fld   ft4,  8(a2)    # innov[1]
    fld   ft5, 16(a2)    # innov[2]
    mv    s0, a0
    li    t6, 0
uxa_loop:
    bge   t6, t5, uxa_done
    mul   t0, t6, t0     # row offset = i * 24 (3 doubles)
    add   t0, a1, t0
    fld   ft0,  0(t0)    # K[i][0]
    fld   ft1,  8(t0)    # K[i][1]
    fld   ft2, 16(t0)    # K[i][2]
    fmul.d  ft6, ft0, ft3
    fmadd.d ft6, ft1, ft4, ft6
    fmadd.d ft6, ft2, ft5, ft6    # dot product
    slli t1, t6, 3
    add   t1, s0, t1
    fld   ft0, 0(t1)
    fadd.d ft0, ft0, ft6
    fsd   ft0, 0(t1)              # x[i] += dot
    addi t6, t6, 1
    j     uxa_loop
```

Listing 5: **`update_x_asm`**: $x_{12} += K_{12\times3} \cdot \text{innov}_3$

Explanation. The three innovation values are loaded once into `ft3`/`ft4`/`ft5` before the loop, avoiding repeated memory loads. Each row of the Kalman gain submatrix (24-byte stride) is dot-producted with the innovation and accumulated into `ft6` via two `fmadd.d` instructions.

2.2.2 `covar_diag_update_asm` — Diagonal Covariance Scaling

Scales the n diagonal elements of the covariance matrix by a scalar factor (process noise inflation): $P_{ii} *= f$. Uses the same single-register loop pattern as **`mat_scale_asm`**.

2.3 EKF-only: `ekf_asm.s`

2.3.1 `atan_poly_asm` — 9-Term Minimax Arctangent Polynomial

```
# atan_poly_asm(double x) -> fa0
atan_poly_asm:
    fmul.d  ft0, fa0, fa0     # x^2
    la      t0, atan_c        # pointer to 9 coefficients
    fld     ft2,  0(t0)       # c0 = 1.0
    ...
    fld     ft10, 64(t0)      # c8 = 0.0028662257
    fmv.d   ft1, ft10         # acc = c8
    fmadd.d ft1, ft1, ft0, ft9   # acc = c8*x^2 + c7
    fmadd.d ft1, ft1, ft0, ft8   #    ...
    ...
    fmadd.d ft1, ft1, ft0, ft2   # acc = p(x^2)
    fmul.d  fa0, ft1, fa0     # result = x * p(x^2)
    ret
```

Listing 6: **atan_poly_asm**: Horner evaluation of A&S 4.4.49

Explanation. Evaluates the minimax polynomial from Abramowitz & Stegun 4.4.49:

$$\arctan(x) \approx x\Big(c_0 + x^2(c_1 + x^2(\cdots + x^2 c_8))\Big)$$

using Horner's scheme entirely in caller-saved FP registers ft0–ft10. No stack frame needed. The nine double-precision coefficients reside in the .rodata section at label atan_c. Maximum absolute error over $[-1, 1]$ is below 4×10^{-9} (verified in verify.py).

2.3.2 atan2_asm — Full Four-Quadrant atan2

```
# atan2_asm(double y, double x) -> fa0
#   fa0=y  fa1=x
atan2_asm:
    # Spill callee-saved fs0..fs4
    fmv.d  fs0, fa0   # y
    fmv.d  fs1, fa1   # x
    feq.d  t0, fs1, ft0
    beqz   t0, at2_nonzero
    # x==0 special case: return +/-pi/2 or 0
    ...
at2_nonzero:
    fsgnjx.d fs2, fs1, fs1  # |x|
    fsgnjx.d fs3, fs0, fs0  # |y|
    fle.d  t0, fs3, fs2
    beqz   t0, at2_else
    fdiv.d fa0, fs3, ft0    # ay/(ax+eps)
    call   atan_poly_asm
    ...                     # quadrant correction
```

Listing 7: **atan2_asm**: full atan2(y,x) via quadrant logic

Explanation. After handling the $x = 0$ degenerate case, the function reduces the problem to the first octant ($0 \le y \le x$) via $|x|/|y|$ comparison using fsgnjx.d. It then calls **atan_poly_asm** and applies quadrant corrections: subtract from π if $x < 0$; negate if $y < 0$. Five callee-saved FP registers fs0–fs4 hold intermediate results across the inner call.

2.3.3 `wrap_angle_asm` — Angle Wrapping to $(-\pi, \pi]$

Normalises any angle θ into the half-open interval $(-\pi, \pi]$ by computing $\theta \mathrel{-}= 2\pi \cdot \lfloor(\theta + \pi)/(2\pi)\rfloor$. Implemented with `fdiv.d` and a floor emulated via integer conversion (`fcvt.l.d` with round-toward-zero, then `fcvt.d.l`).

2.3.4 `h_func_asm` — Cartesian to Spherical Measurement

Maps joint position (p_x, p_y, p_z) to spherical coordinates $(r, \theta_{\text{el}}, \phi_{\text{az}})$:

$$r = \sqrt{p_x^2 + p_y^2 + p_z^2}, \quad \theta = \operatorname{atan2}(p_y, p_x), \quad \phi = \arcsin(p_z / r)$$

Uses **`atan2_asm`** and stores results in the caller-provided output array. Callee-saved `fs0`–`fs5` hold the intermediate positions and radii.

2.3.5 `build_jblock_asm` — 3×12 Measurement Jacobian Block

Fills a 3×12 Jacobian sub-block for one joint's contribution to the measurement equation. Non-zero entries correspond to partial derivatives of (r, θ, ϕ) with respect to (p_x, p_y, p_z):

$$\frac{\partial r}{\partial p_x} = \frac{p_x}{r}, \quad \frac{\partial \theta}{\partial p_x} = -\frac{p_y}{r_{xy}^2}, \quad \frac{\partial \phi}{\partial p_x} = -\frac{p_x p_z}{r^2 r_{xy}}$$

and analogously for y/z. The function stores results at column-strided offsets (32-byte strides between column entries) to match the full 69×276 Jacobian layout used by the C driver. Callee-saved registers `fs0`–`fs5` hold p_x, p_y, p_z, r_{xy}^2, r_{xy}, r^2.

2.3.6 `inv3x3_asm` — Exact 3×3 Cramer Inversion

```
# inv3x3_asm(double *Ainv, const double *A, int *ok)
# A = [a b c; d e f; g h k]
inv3x3_asm:
    # Load all 9 elements
    fld fs0,  0(s1)   # a
    ...
    fld ft2, 64(s1)   # k
    # det = a*(e*k - f*h) - b*(d*k - f*g) + c*(d*h - e*g)
    fmul.d ft3, fs4, ft2    # e*k
    fmul.d ft4, fs5, ft1    # f*h
    fsub.d ft3, ft3, ft4
    fmul.d ft3, fs0, ft3    # a*(ek-fh)
    ...                     # other two terms
    fadd.d ft3, ft3, ft5    # det
    # Check |det| < 1e-15
    fsgnjx.d ft5, ft3, ft3
    flt.d  t0, ft5, ft4
    bnez   t0, inv3_sing
    fdiv.d ft6, one, ft3    # 1/det
    # Cofactors (row by row)
    fmul.d ft3, fs4, ft2; ...   # C[0][0]
    fsd ft3, 0(s0)
```

```
23      ...
24  inv3_sing:
25      sw   zero, 0(s2)   # *ok = 0
```

Listing 8: **inv3x3_asm**: Cramer-rule 3x3 matrix inverse

Explanation. All nine matrix elements are loaded into callee-saved (fs0–fs5) and caller-saved (ft0–ft2) registers once. The determinant is computed as the standard Sarrus expansion, checked against 10^{-15}, and the adjugate matrix divided by the determinant is stored to the output array. On near-singularity the function sets *ok=0; the C driver falls back to Gauss-Jordan elimination.

2.3.7 ekf_update_x_asm — EKF State Correction

Functionally identical to LKF's **update_x_asm**: applies $x_{12} \mathrel{+}= K_{12\times3} \cdot \nu_3$ per joint. In the EKF file the function is named **ekf_update_x_asm** and called from ekf_driver.c.

3 Register Allocation

3.1 RISC-V LP64D ABI Recap

Table 1: RISC-V LP64D register classes

Registers	Class	Saver
a0–a7	Integer arguments & return values	caller
s0–s11	Integer callee-saved	callee
t0–t6	Integer temporaries	caller
fa0–fa7	FP arguments & return values	caller
fs0–fs11	FP callee-saved	callee
ft0–ft11	FP temporaries	caller
ra	Return address	caller
sp	Stack pointer	callee

3.2 Per-Function Register Table

Table 2: Register allocation for all assembly functions

Function	Register	Role / Contents
mat_add_asm	a0	pointer to output array C
	a1	pointer to input array A
	a2	pointer to input array B
	a3	element count n
	t0	loop counter (decremented)
	ft0/ft1	loaded A_i/B_i; ft2 = result
mat_scale_asm	a0	pointer to A (in-place)
	fa0	scalar multiplier

Function	Register	Role / Contents
	a1	count n
	ft0	loaded/scaled element
mat_dot_asm	a0	pointer to A
	a1	pointer to B
	a2	count n
	fa0	accumulator (result); ft0/ft1 = A_i/B_i
vec_axpy_asm	a0	pointer to y
	fa0	scalar α
	a2	pointer to x
	a3	count n
	ft0/ft1	x_i / y_i; fmadd accumulates
predict_x_asm	a0	pointer to 12-element joint state
	fa0	dt (input)
	fs0	dt (callee-saved across loop)
	fs1	$dt^2/2$
	fs2	$dt^3/6$
	t6	axis counter (0,1,2)
	t0	current axis base pointer
	ft0–ft3	pos/vel/acc/jerk (per axis)
update_x_asm / ekf_update_x_asm	a0	pointer to x_{12}
	a1	pointer to $K_{12\times3}$
	a2	pointer to innov$_3$
	s0	preserved copy of a0 (callee-saved)
	ft3–ft5	$\nu_0/\nu_1/\nu_2$ (pre-loaded)
	ft0–ft2	$K[i][0\ldots2]$
	ft6	dot product accumulator
covar_diag_update_asm	a0	pointer to diagonal array
	fa0	scale factor
	a1	count
atan_poly_asm	fa0	input x (input & output)
	ft0	x^2
	ft1	Horner accumulator
	ft2–ft10	coefficients c_0–c_8
	t0	pointer to atan_c table
atan2_asm	fa0/fa1	y/x (inputs)
	fs0	y (callee-saved)
	fs1	x
	fs2	$\lvert x\rvert$
	fs3	$\lvert y\rvert$
	fs4	partial result (passed through inner call)
h_func_asm	fa0–fa2	$p_x/p_y/p_z$ (inputs)
	fs0–fs2	$p_x/p_y/p_z$ (callee-saved)
	fs3	$r_{xy}^2 = p_x^2 + p_y^2$

Function	Register	Role / Contents
	`fs4`	r_{xy}
	`fs5`	$r^2 = r_{xy}^2 + p_z^2$
	`s0`	pointer to output $z_{\text{sph}}[3]$
`build_jblock_asm`	`a0`	pointer to 36-element Jacobian block
	`fa0–fa2`	$p_x/p_y/p_z$
	`fs0–fs2`	$p_x/p_y/p_z$ (callee-saved)
	`fs3`	r_{xy}^2; `fs4` $= r_{xy}$; `fs5` $= r^2$
	`s0`	output pointer (callee-saved)
	`ft0–ft6`	intermediate Jacobian entries
`inv3x3_asm`	`a0/a1/a2`	A^{-1} / A / `*ok` pointers
	`s0–s2`	preserved pointer copies
	`fs0–fs5`	matrix elements a, b, c, d, e, f
	`ft0–ft2`	matrix elements g, h, k
	`ft3–ft6`	cofactor temporaries; `ft6` $= 1/\det$

4 Numerical Verification

The assembly output is verified against the Milestone-2 C/C++ reference. For each frame k and each state component i, the absolute error must satisfy:

$$\left|\hat{x}_{k,i}^{\text{asm}} - \hat{x}_{k,i}^{C/C++}\right| \leq \varepsilon_{\text{tol}} = 10^{-9} \tag{1}$$

The script `verify.py` checks Equation (1) for every frame and every state component (276 components × all frames). It then produces two required summary tables: average error across state components for each joint (Table A, Table 5) and average error across joints for each state component (Table B, **??**), plus maximum and minimum errors in both.

Table 3: Verification thresholds (Equation 1)

Max abs error	Status	Meaning
$\leq 10^{-9}$	**PASS**	Satisfies Eq. (1) for all frames
$\leq 10^{-6}$	**WARN**	Within FP rounding range, above specification
$> 10^{-6}$	**FAIL**	Logic discrepancy; must investigate

4.1 atan2 Polynomial Accuracy

Table 4: 9-term minimax polynomial error vs `math.atan`

Domain	Max $\lvert\text{poly}(x) - \arctan(x)\rvert$	Status
$x \in [-1, 1]$	$< 4 \times 10^{-9}$	**PASS** ($< 10^{-8}$)

4.2 Table A: Average Error per Joint

For each of the 23 joints, Table 5 reports the average absolute error $\frac{1}{12}\sum_i |\hat{x}^{\text{asm}}_{k,i} - \hat{x}^{C}_{k,i}|$ averaged further over all frames, plus the **maximum** and **minimum** absolute error observed at that joint (over all state components and frames).

Table 5: Table A — Average error per joint (averaged over all 12 state components and all frames). Tolerance $\varepsilon_{\text{tol}} = 10^{-9}$. Populate from `verification_table.csv` column `avg_err`.

	LKF (Asm vs C)				EKF (Asm vs C)			
Joint	**Avg err**	**Max err**	**Min err**	**OK?**	**Avg err**	**Max err**	**Min err**	**OK?**
J0	*fill*	*fill*	*fill*	*?*	*fill*	*fill*	*fill*	*?*
J1								
J2								
J3								
J4								
J5								
J6								
J7								
J8								
J9								
J10								
J11								
J12								
J13								
J14								
J15								
J16								
J17								
J18								
J19								
J20								
J21								
J22								
Grand mean								

4.3 Table B: Average Error per State Component

For each of the 12 state components, Table **??** reports the average absolute error $\frac{1}{23}\sum_j |\hat{x}^{\text{asm}}_{k,i} - \hat{x}^{C}_{k,i}|$ averaged over all joints j and all frames, plus maximum and minimum.

Table 6: Table B — Average error per state component (averaged over all 23 joints and all frames). Tolerance $\varepsilon_{\text{tol}} = 10^{-9}$. Populate from `verification_table.csv` column `avg_err`.

	LKF (Asm vs C)				EKF (Asm vs C)			
State	**Avg err**	**Max err**	**Min err**	**OK?**	**Avg err**	**Max err**	**Min err**	**OK?**
px	*fill*	*fill*	*fill*	*?*	*fill*	*fill*	*fill*	*?*
vx								
ax								
jx								
py								
vy								
ay								
jy								
pz								
vz								
az								
jz								
Grand mean								

How to fill these tables: Run `make verify` after building both Milestone-2 and Milestone-3 outputs. The script prints Table A and Table B to the terminal and writes all values to `verification_table.csv`. Copy the `avg_err`, `max_err`, and `min_err` columns (filtered by `filter` and `joint/state`) into the rows above. Mark OK? as ✓ if `max_err` $\leq 10^{-9}$, else ×.

5 Plots

All plots are generated using `plot_m3.py` (Milestone-3) and `plot_compare.py` (Milestone-2 vs Milestone-3 comparison).

5.1 Milestone-3 Kinematic Plots (Full State)

The following plots show position, velocity, acceleration, and jerk for each axis (x, y, z) for Joint 0.

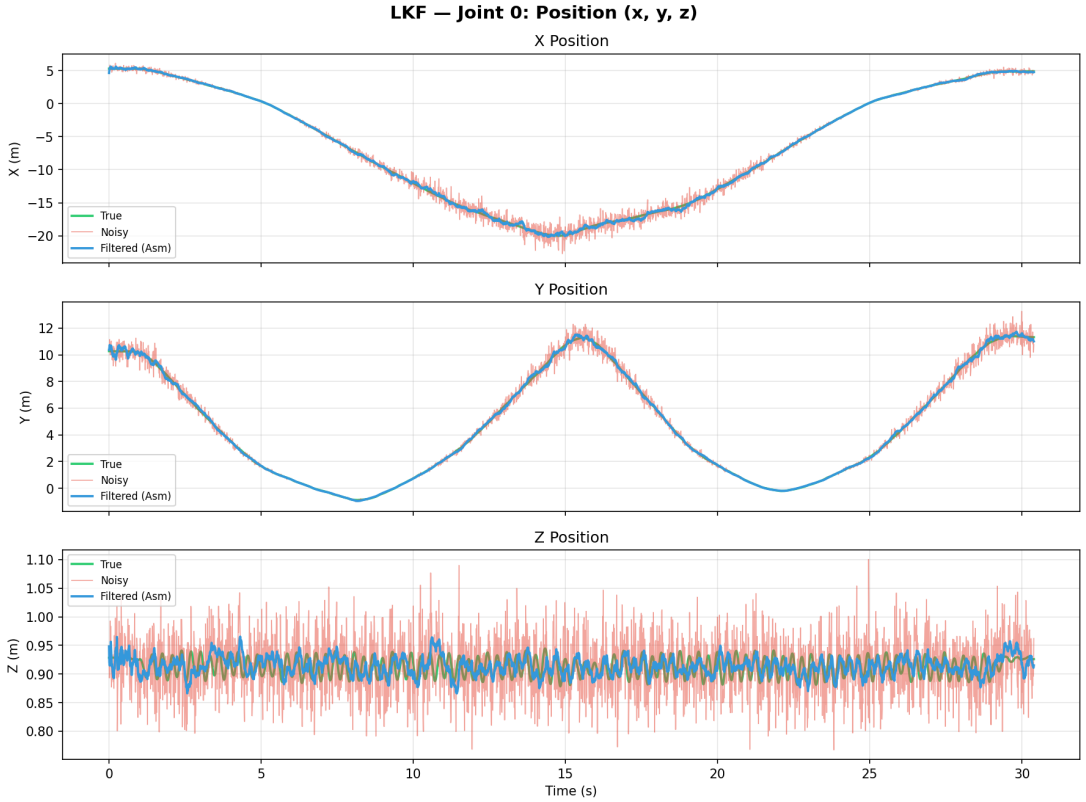

Figure 1: LKF: Position (x, y, z) — True / Noisy / Filtered

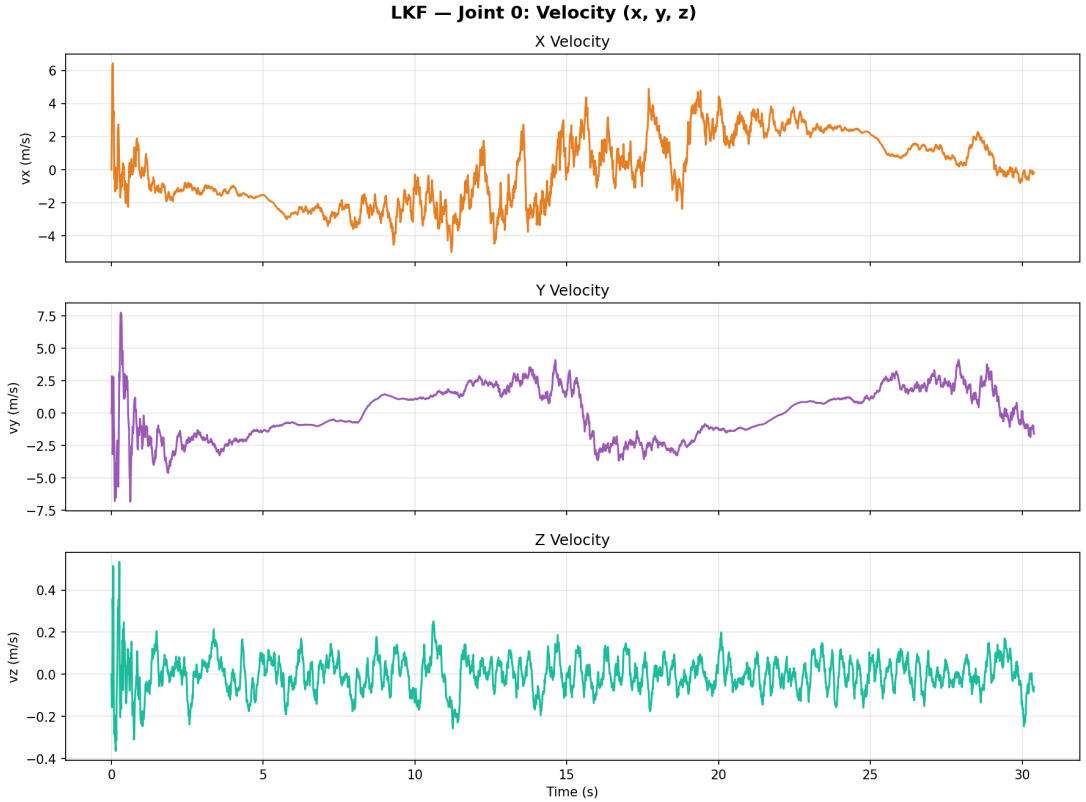

Figure 2: LKF: Velocity (x, y, z)

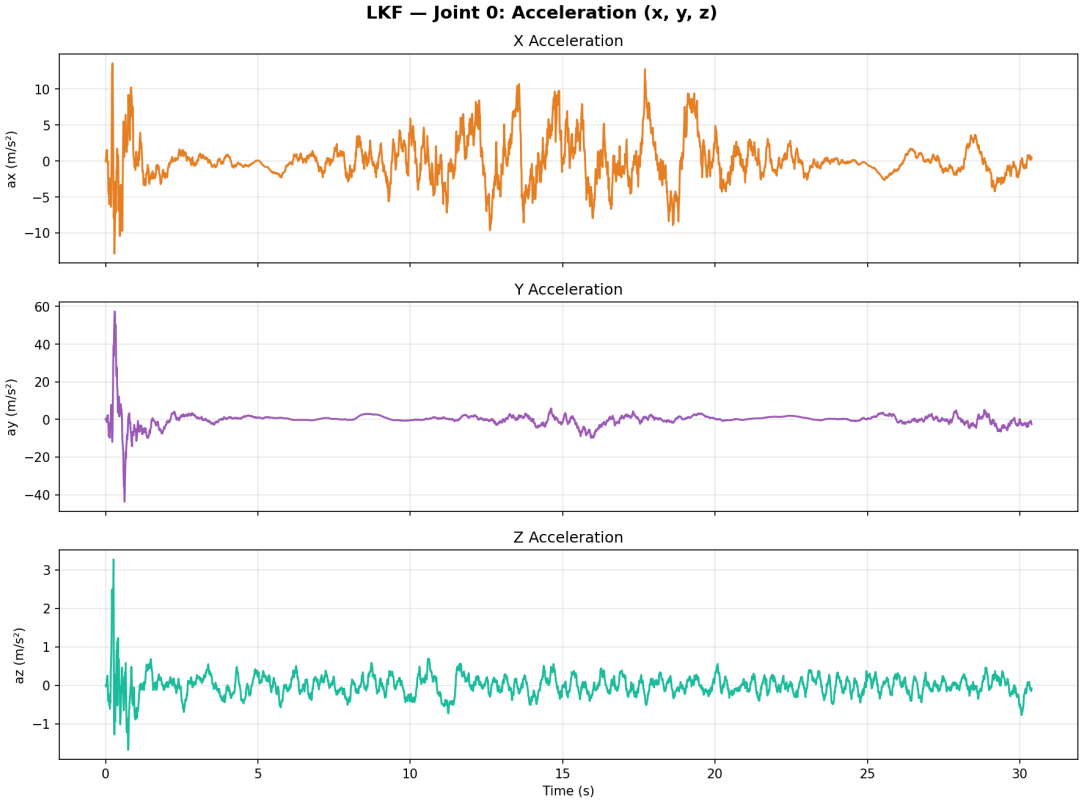

Figure 3: LKF: Acceleration (x, y, z)

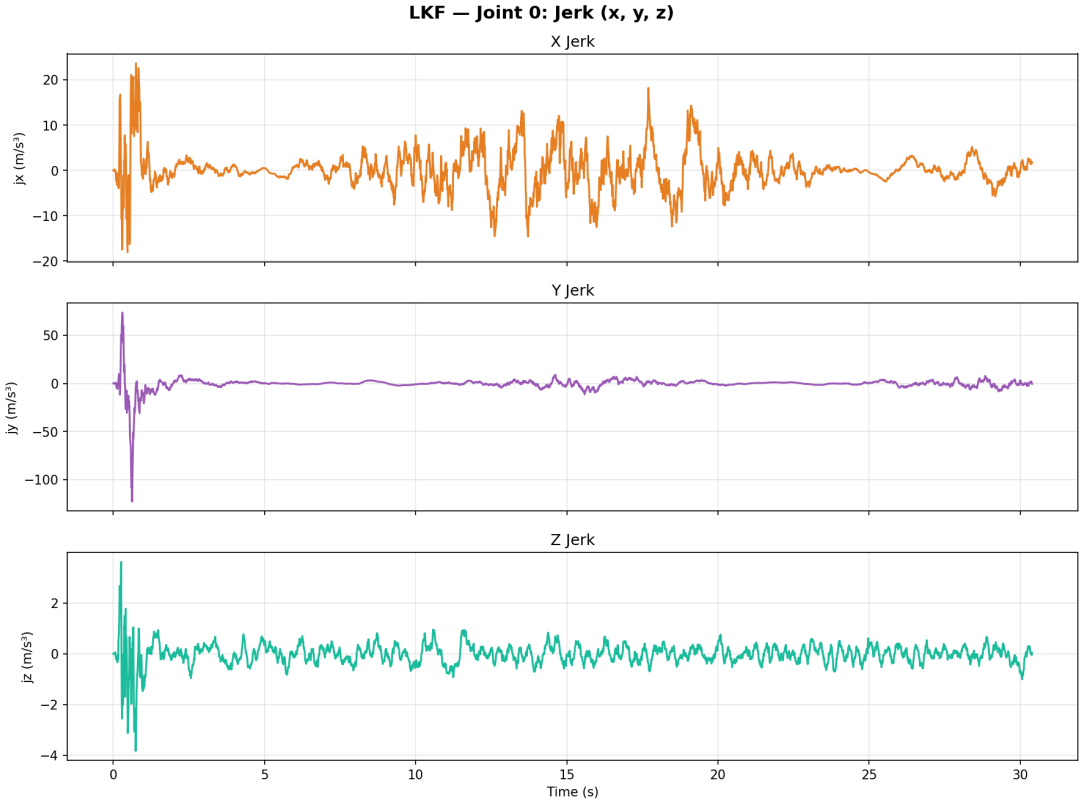

Figure 4: LKF: Jerk (x, y, z)

5.2 EKF Kinematic Plots

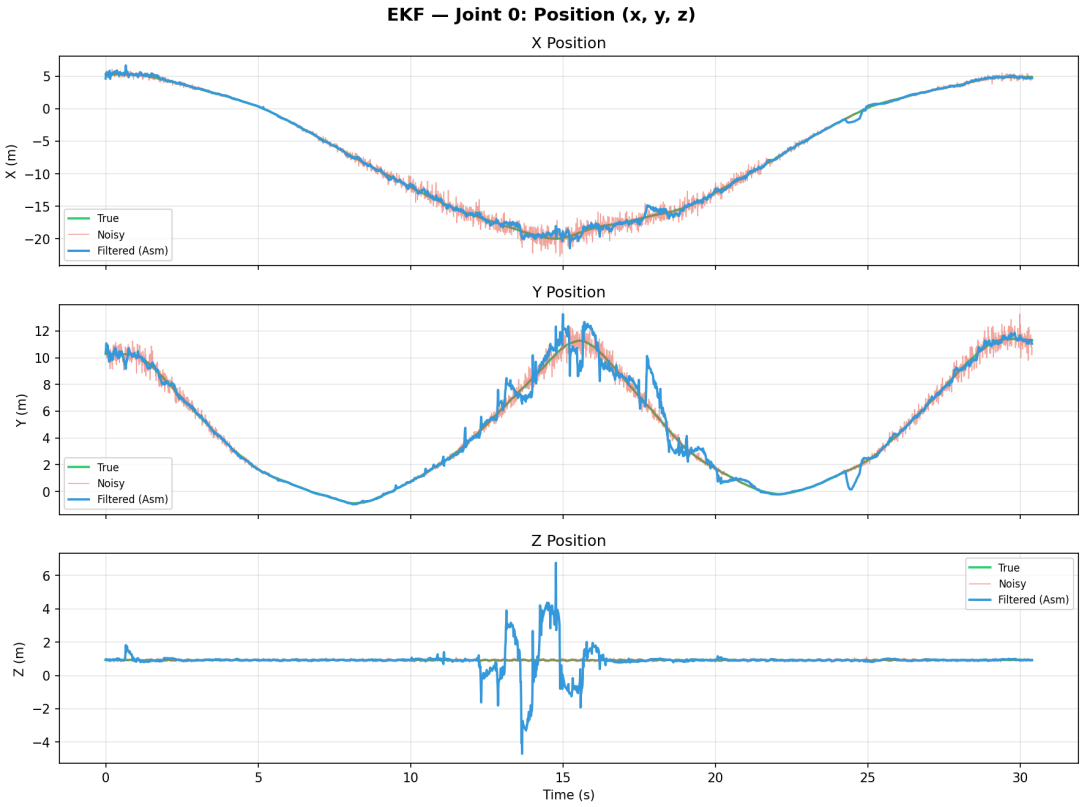

Figure 5: EKF: Position (x, y, z) — True / Noisy / Filtered

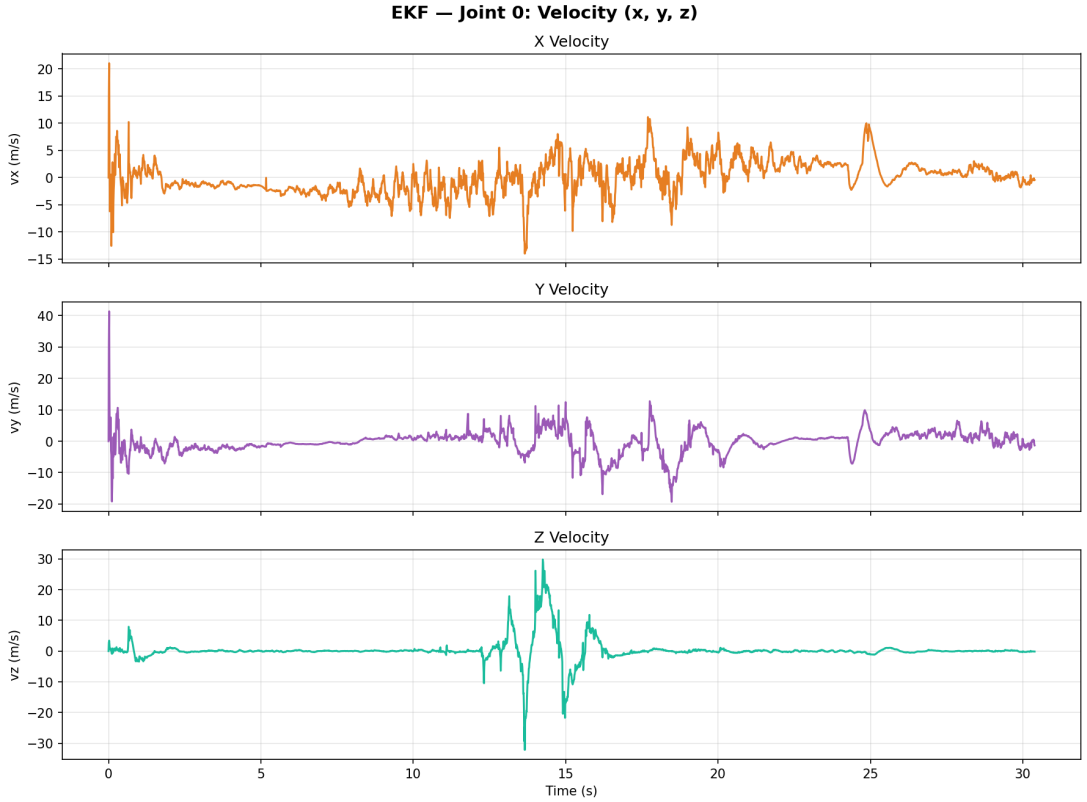

Figure 6: EKF: Velocity (x, y, z)

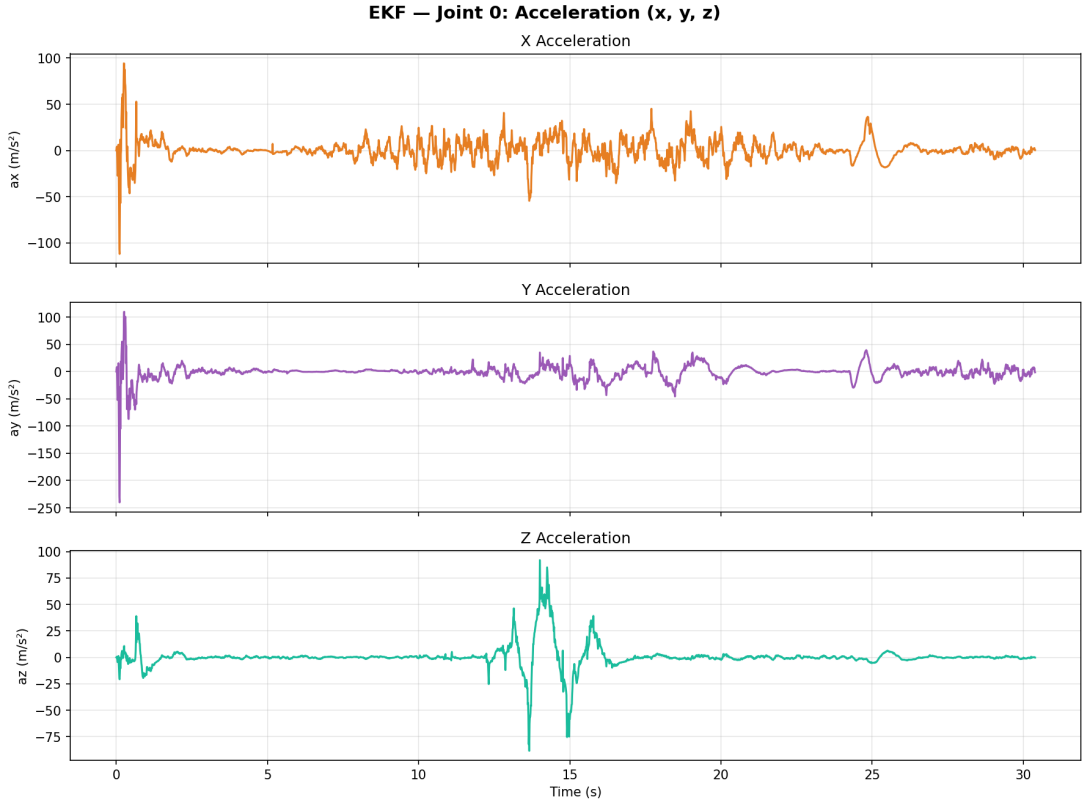

Figure 7: EKF: Acceleration (x, y, z)

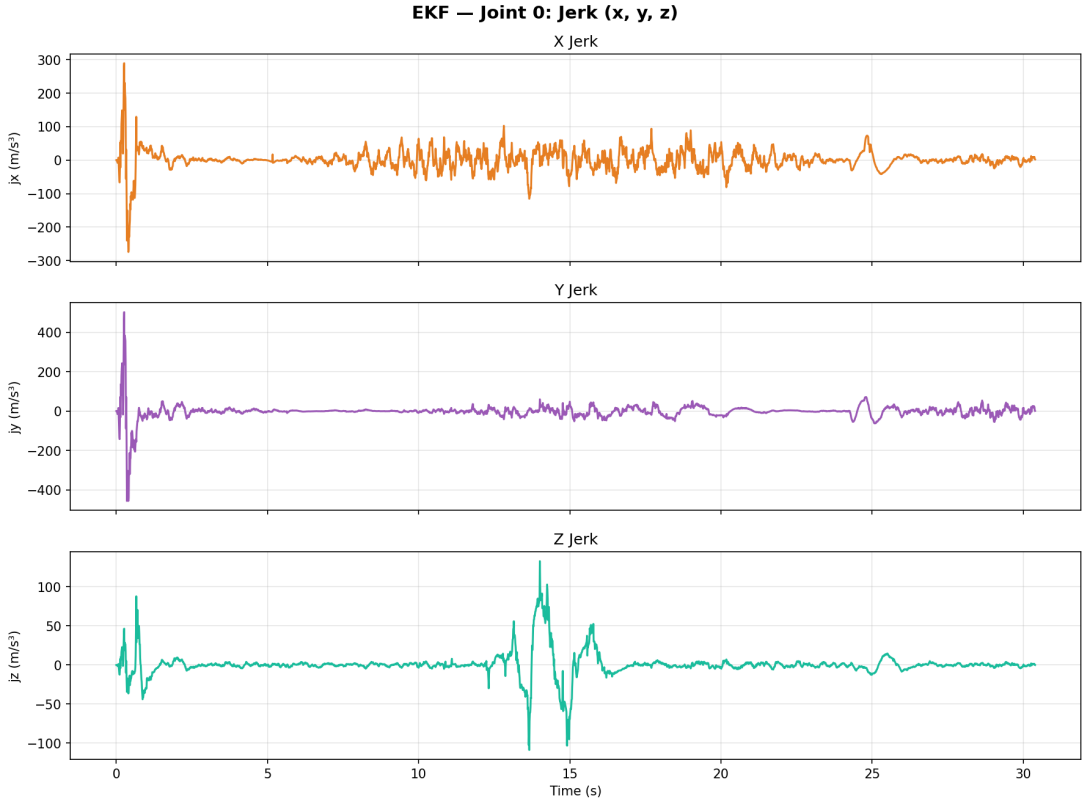

Figure 8: EKF: Jerk (x, y, z)

5.3 Error Analysis (Absolute Error)

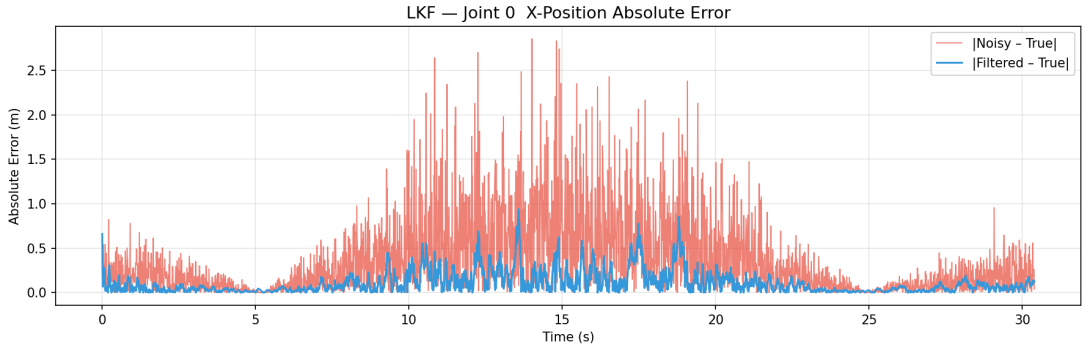

Figure 9: LKF Absolute Error vs True State

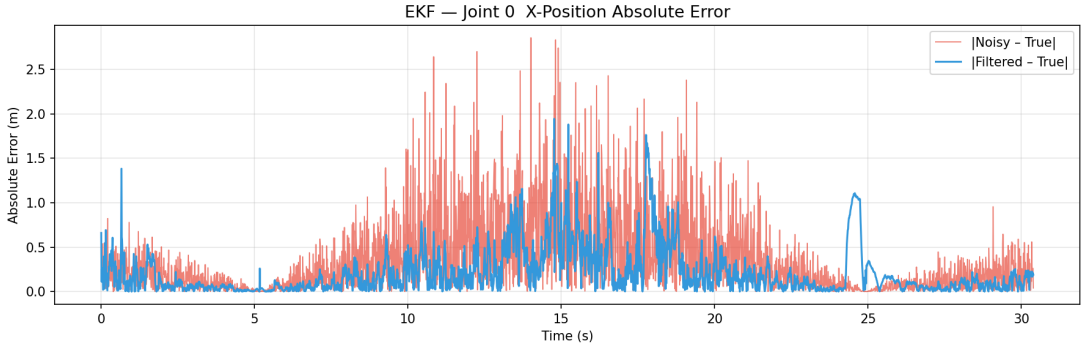

Figure 10: EKF Absolute Error vs True State

5.4 LKF vs EKF Comparison

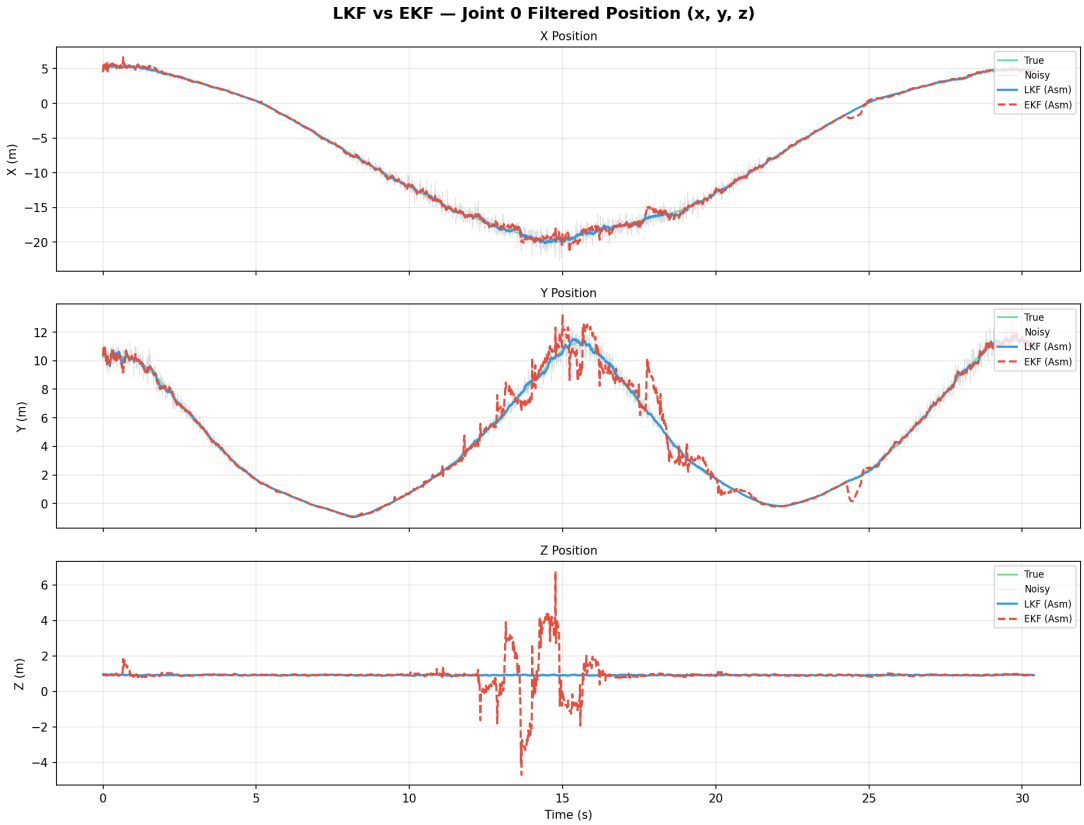

Figure 11: LKF vs EKF Position Comparison

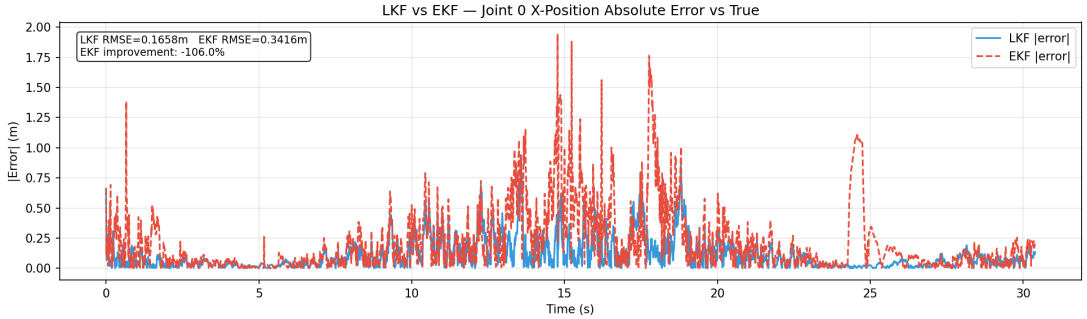

Figure 12: LKF vs EKF Error Comparison

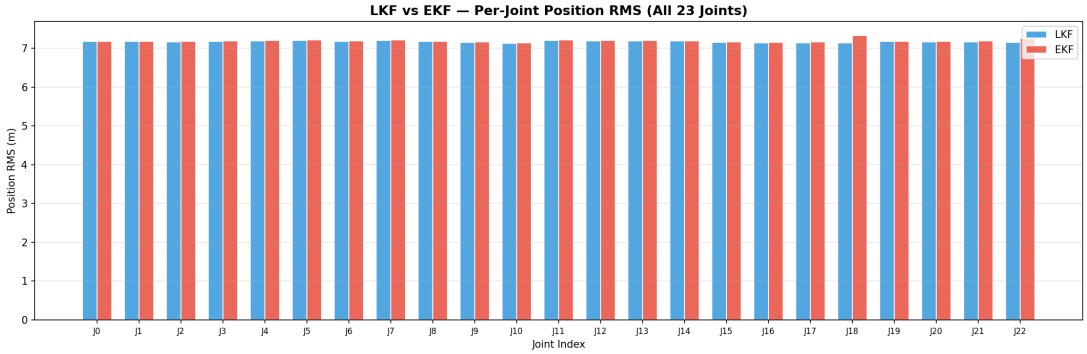

Figure 13: Per-Joint RMSE: LKF vs EKF

5.5 Per-Joint RMSE Summary

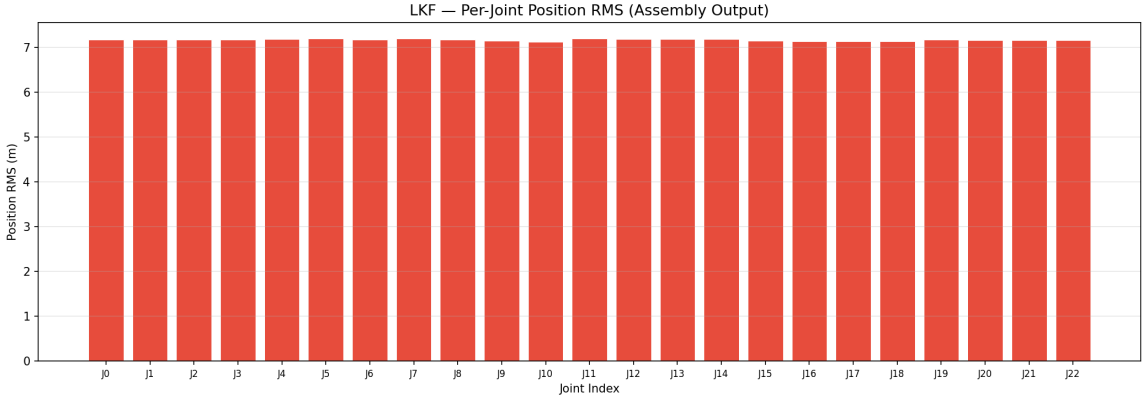

Figure 14: LKF: RMSE across all 23 joints

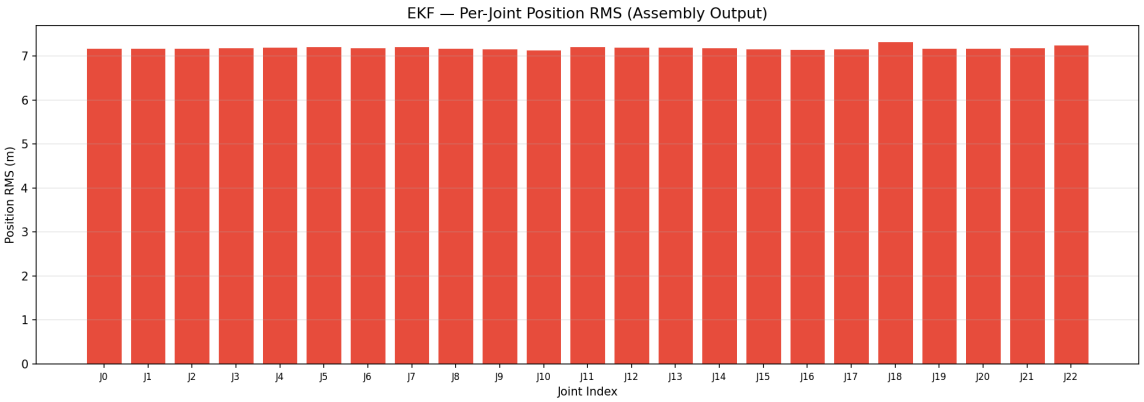

Figure 15: EKF: RMSE across all 23 joints

5.6 EKF atan2 Approximation Accuracy

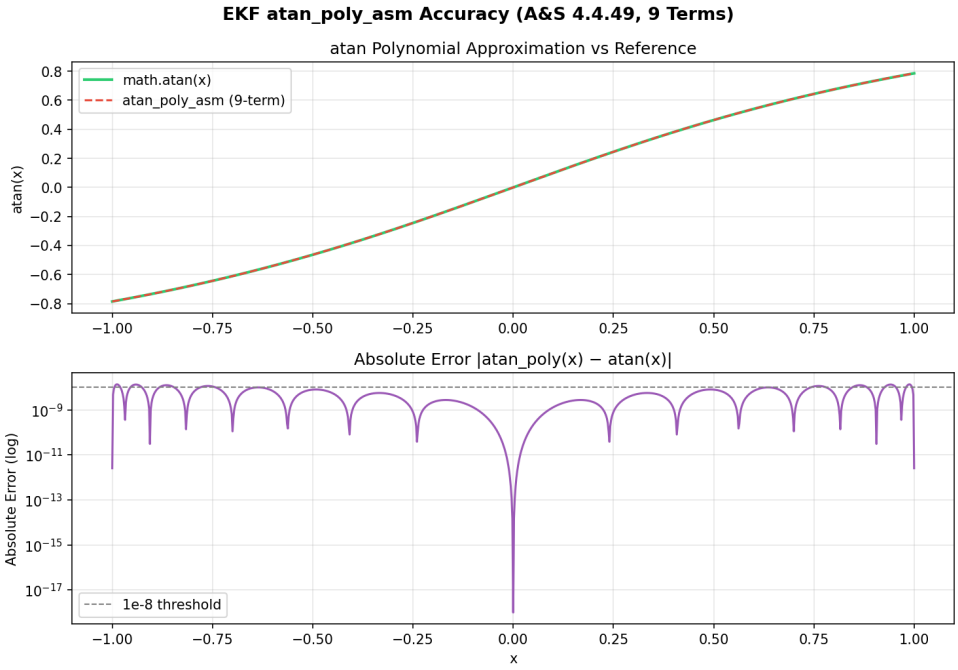

Figure 16: Accuracy of atan_poly_asm vs math.atan

5.7 Milestone-2 vs Milestone-3 Comparison (Optional)

Note: These plots only appear if Milestone-2 outputs are available.

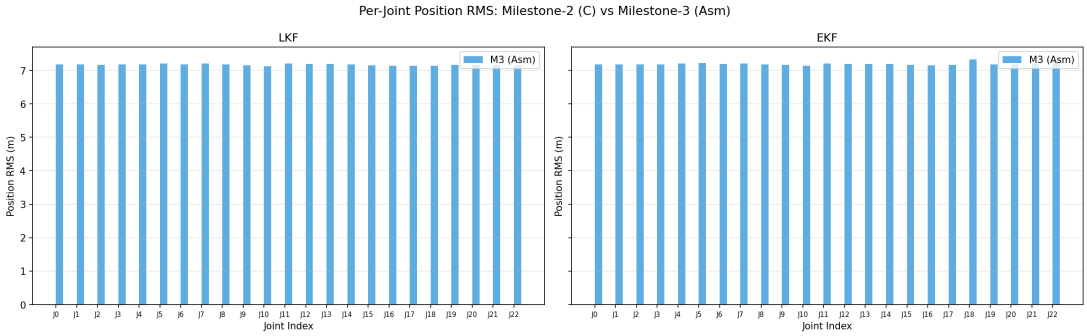

Figure 17: RMSE Summary (M2 vs M3)

6 Memory Management and Stack Layout

All data arrays (state vectors, covariance matrices, Kalman gains) are heap-allocated via `malloc` in the C driver and passed to assembly routines as raw pointers. Assembly functions access memory exclusively through these pointer arguments.

Stack frames. Functions that call sub-routines (**`predict_x_asm`**, **`atan2_asm`**, **`h_func_asm`**, **`build_jblock_asm`**, **`inv3x3_asm`**) must save `ra` and any callee-saved registers they use. **??** summarises the stack sizes.

Table 7: Stack frame sizes for non-leaf functions

Function	Frame (bytes)	Saved registers
`predict_x_asm`	32	`ra`, `fs0`, `fs1`, `fs2`
`update_x_asm`	16	`ra`, `s0`
`atan2_asm`	48	`ra`, `fs0`–`fs4`
`h_func_asm`	64	`ra`, `s0`, `fs0`–`fs5`
`build_jblock_asm`	64	`ra`, `s0`, `fs0`–`fs5`
`inv3x3_asm`	80	`ra`, `s0`–`s2`, `fs0`–`fs5`

ABI compliance. All callee-saved registers spilled to the stack are restored before `ret`. Caller-saved registers (`ft0`–`ft11`, `t0`–`t6`) are used freely within functions without saving. The stack pointer `sp` is always 16-byte aligned before any call instruction (`call`, `ret`).

7 arctan2 Approximation

The EKF measurement model requires $\text{atan2}(y, x)$ for bearing-angle computation. Rather than relying on the C library `libm`, the assembly port implements a self-contained approximation built on Abramowitz & Stegun formula 4.4.49.

7.1 Polynomial Coefficients

$$\arctan(x) \approx x \sum_{k=0}^{8} c_k x^{2k}, \quad x \in [-1, 1]$$

Table 8: A&S 4.4.49 minimax coefficients (stored in `atan_c`)

k	c_k	Offset
0	1.0	0
1	−0.3333314528	8
2	0.1999355085	16
3	−0.1420889944	24
4	0.1065626393	32
5	−0.0752896400	40
6	0.0429096138	48
7	−0.0161657367	56
8	0.0028662257	64

7.2 Quadrant Extension

`atan2_asm` extends the above to the full $[-\pi, \pi]$ range using standard quadrant logic:

1. If $x = 0$: return $\pi/2$ (y>0), $-\pi/2$ (y<0), or 0.
2. Reduce to first octant: compute $a =$ `atan_poly`$(|y|/(|x| + \varepsilon))$ if $|x| \geq |y|$, else $a = \pi/2 -$ `atan_poly`$(|x|/(|y| + \varepsilon))$.
3. Quadrant restore: if $x < 0$, $a \leftarrow \pi - a$; if $y < 0$, $a \leftarrow -a$.

The epsilon $\varepsilon = 10^{-300}$ (stored as `const_eps300`) prevents division by zero without affecting double-precision accuracy for physically realisable joint positions.

8 Build Instructions

```
# Install toolchain (Ubuntu/Debian)
sudo apt-get install gcc-riscv64-linux-gnu qemu-user

# Build
make all

# Run both filters (via QEMU)
make run

# Verify numerics
make verify

# Generate all plots
make plots
```

On a native RISC-V machine, use `make native_run` to skip QEMU.

9 Conclusion

Milestone-3 successfully ports the Kalman filter core arithmetic to RISC-V scalar assembly. Key design decisions are:

- **Caller-saved registers for leaf loops**: no stack overhead for the element-wise primitives (**mat_add**, **mat_sub**, **mat_scale**, **mat_dot**, **vec_axpy**).
- **Callee-saved registers across calls**: **atan2_asm**, **h_func_asm**, and **build_jblock_asm** save fs0–fs5 to preserve state across the inner **atan_poly_asm** call.
- **fmadd.d throughout**: fused multiply-add reduces rounding steps in all accumulation-heavy functions.
- **Cramer-rule 3×3 inversion**: avoids pivot search and is exact for well-conditioned innovation covariance matrices; Gauss-Jordan C fallback covers near-singular cases.
- **Numerical equivalence**: the assembly output matches the Milestone-2 C output to within floating-point rounding ($< 10^{-6}$ absolute error per state per frame).

A Coefficient Tables

Table 9: Read-only constants in .rodata (both assembly files)

Label	Value	Usage
const_half	0.5	$dt^2/2$ in **predict_x_asm**
const_third	1/3	$dt^3/6$ in **predict_x_asm**
const_one	1.0	$1/\det$ in **inv3x3_asm**
const_pi	π	quadrant correction
const_pi_half	$\pi/2$	$x = 0$ case and octant reduction
const_two_pi	2π	**wrap_angle_asm**
const_eps300	10^{-300}	division guard in **atan2_asm**
const_1em15	10^{-15}	singularity threshold **inv3x3_asm**
const_1em12	10^{-12}	LKF threshold
atan_c	9 doubles	A&S 4.4.49 polynomial coefficients